ALGORITHMS ANALYSIS AND DESIGN

Lekha A
Computer Applications

PES UNIVERSITY
CELEBRATING 50 YEARS

ALGORITHMS ANALYSIS AND DESIGN

Introduction, Analysis Framework and Sorting Techniques

Sorting Algorithms – Divide and Conquer

Lekha A
Computer Applications

ALGORITHMS ANALYSIS AND DESIGN
Quick sort

- It was invented by a British scientist C.A.R Hoare.

- Quick sort divides the elements according to their values.

Figure 2: https://en.wikipedia.org/wiki/Tony_Hoare

ALGORITHMS ANALYSIS AND DESIGN
Quick Sort

PES UNIVERSITY
CELEBRATING 50 YEARS

- Quick Sort uses the Divide and Conquer paradigm to sort a given list of numbers.
- The execution of Quick Sort typically follows three key steps
  - Divide
  - Conquer
  - Combine

ALGORITHMS ANALYSIS AND DESIGN
Quick Sort - Divide

PES UNIVERSITY
CELEBRATING 50 YEARS

- The algorithm starts with the division process.
  - It involves the selection of a pivot element from the array.
- The pivot element is used to divide the array into two segments.
- The aim of this division is to ensure that the array is rearranged in such a way that all elements that are less than the pivot element are placed before it and all elements that are greater than the pivot element are placed after it.

ALGORITHMS ANALYSIS AND DESIGN

Quick Sort - Conquer

- Once the partitioning is finished, the pivot element will be in its correct position in the array.
- Once the array is partitioned, the algorithm enters the conquer phase.
- This phase includes the two partitioned arrays created from the elements that lie before and after the pivot and the recursive application of the same Quick Sort algorithm.

ALGORITHMS ANALYSIS AND DESIGN
Quick Sort - Conquer

PES UNIVERSITY
CELEBRATING 50 YEARS

- For each of these sub arrays, a new pivot is chosen and the sub array is partitioned around the pivot.
- The recursive application of the algorithm continues until the base case is reached.
  - This is when the sub array has only one element.

ALGORITHMS ANALYSIS AND DESIGN
Quick Sort - Combine

- Quick Sort does not require a combine step.
- It is an in-place sorting algorithm.
- It does not require the creation of a new array to hold the sorted elements but rather holds the elements in the original array itself.

ALGORITHMS ANALYSIS AND DESIGN
Quick Sort - Combine

- This means that once the conquer step has sorted the sub-arrays, the original array is already sorted.
- This means that there is no need to combine the sorted sub-arrays.

ALGORITHMS ANALYSIS AND DESIGN

Quick Sort - Methodology

- The core operation of Quick Sort lies in the selection of a pivot element from the array.
- The remainder of the array is then divided into two distinct sections based on this pivot, those elements which are less than the pivot and those which are greater.
- The selection of the pivot element is not a trivial task as it can greatly impact the efficiency of the sorting process.

ALGORITHMS ANALYSIS AND DESIGN

Quick Sort - Methodology

- An ideal pivot would divide the array into two nearly equal halves, leading to an optimal Quick Sort operation.
- Once the array has been partitioned based on the pivot, the pivot element will find its accurate place in the final sorted array.
- It separates all elements smaller than it on its left side and all elements larger on its right side.

ALGORITHMS ANALYSIS AND DESIGN
Quick Sort - Methodology

- After partitioning around the pivot, the algorithm is reapplied to each of the two partitions, elements less than the pivot and elements greater than the pivot.

- This process repeats until the sub array has one element.

$$\underbrace{A[0]\ldots A[s-1]}_{all\ are \leq A[s]}\ A[s]\ \underbrace{A[s+1]\ldots A[n-1]}_{all\ are > A[s]}$$

ALGORITHMS ANALYSIS AND DESIGN

Quick Sort - Steps

- Step 1
  - Select a pivot. This could be the first, middle, or last element of the array or a random element.
- Step 2
  - Partition the array around the pivot.
  - Move elements smaller than the pivot before it and elements larger than it after.

Quick Sort - Steps

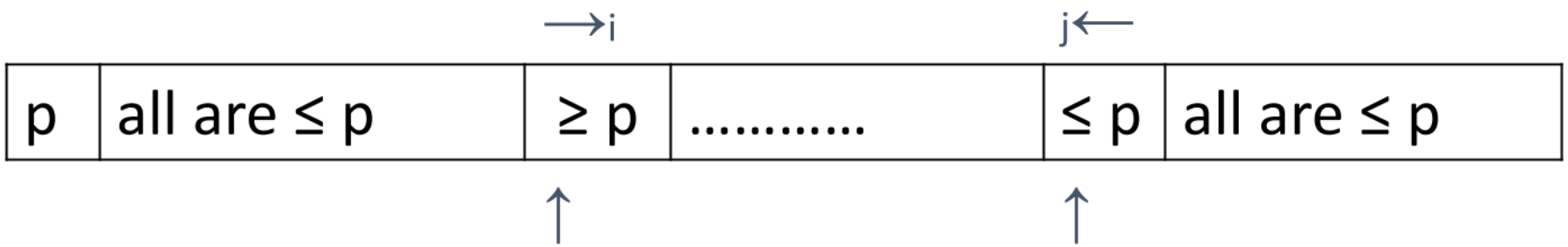
- Step 2 (Left to right scan)
  - If A[i] ≤ pivot, increment i by 1.
    - It starts with the second element.
    - It skips all elements smaller than the pivot and stops when a bigger or equal element is found.

Quick Sort - Steps

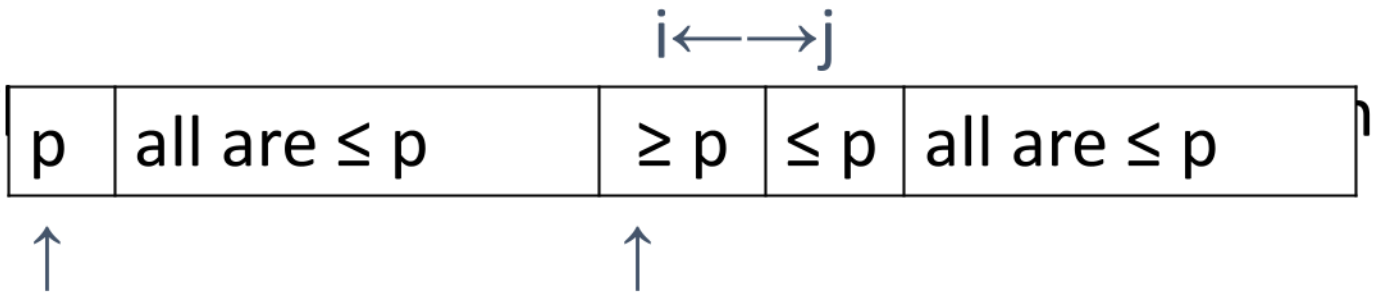
- Step 2 (Right to left scan)
  - If A[j] > pivot, decrement j by 1.
    - It starts with the last element of the sub array.
    - It skips all elements bigger than the pivot and stops when a smaller or equal element is found.

ALGORITHMS ANALYSIS AND DESIGN
Quick Sort - Steps

- Step 2 (Right to left scan)
  - If i < j
    - i and j have not crossed.
    - Exchange A[i] and A[j] and continue.



ALGORITHMS ANALYSIS AND DESIGN
Quick Sort - Steps

- Step 2 (Right to left scan)
  - If i > j
    - i and j have crossed over.
    - T 

nging the pivot with $A_{[j]}$.

Quick Sort - Steps

- Step 3
  - Repeat these steps for the sub arrays on both sides of the pivot.
  - This is a recursive operation.

ALGORITHMS ANALYSIS AND DESIGN
Algorithm – Partition(A[l..r])

- // Partitions a sub array by using its first element as a pivot.
- // Input : A Sub array A[l….r] of A[0….n-1], defined by its left and right //indices l and r (l<r).
- // Output : A partition of A[l…r], with the split position returned as this //function’s value.

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Partition(A[l..r])

p← A[l]

i ←l+1; j ← r

while (true)

repeat i ← i+1 until A[i]< p

repeat j ← j-1 until A[j] ≥ p

swap (A[i], A[j])

until i ≥ j

swap (A[i],A[j]) //undo last swap when i ≥ j

swap(A[l], A[j])

return j

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Quicksort(A[l..r])

- // Sorts a subarray by quicksort
- // Input: Subarray of array A[0..n − 1], defined by its left and right indices l

  //and r
- // Output: Subarray A[l..r] sorted in nondecreasing order

ALGORITHMS ANALYSIS AND DESIGN
Algorithm – Quicksort(A[l..r])

PES UNIVERSITY
CELEBRATING 50 YEARS

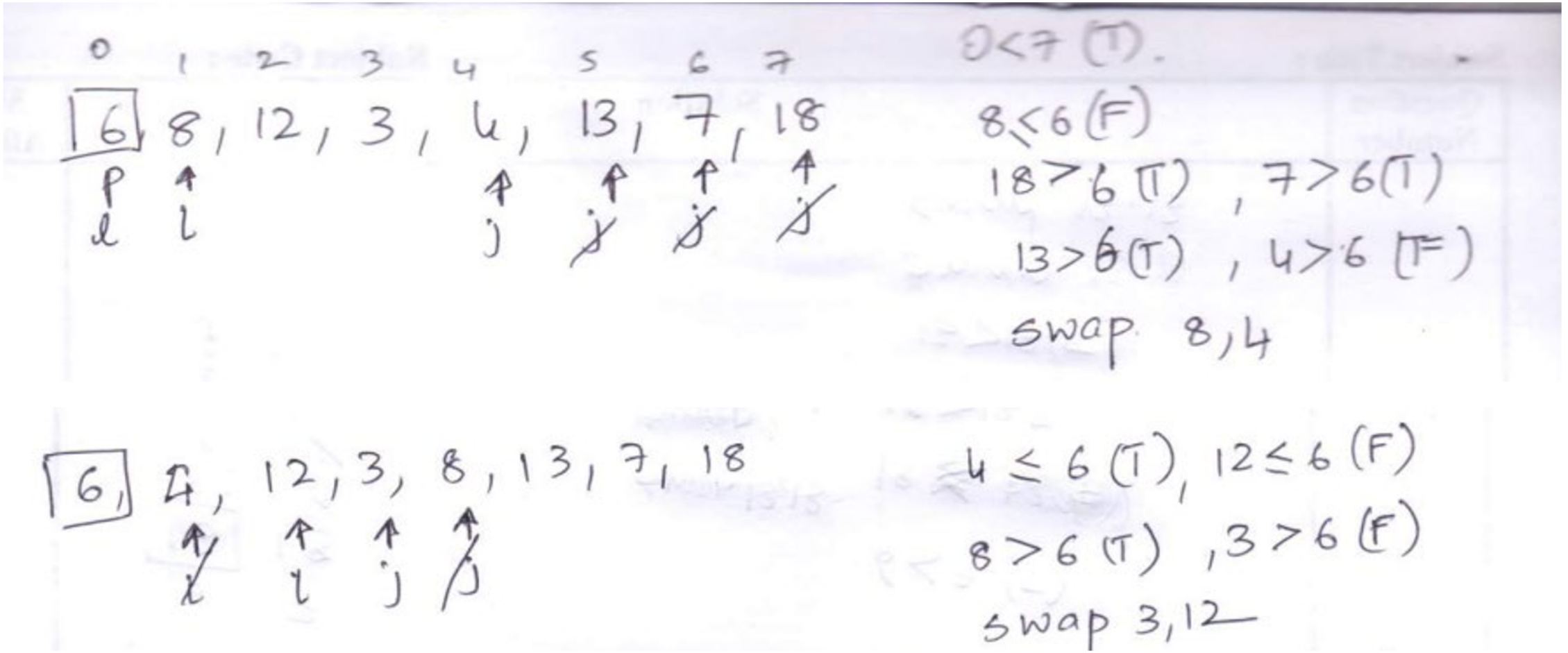
- if l < r

  s ← Partition(A[l..r]) // s is a split position

  Quicksort(A[l..r − 1])

  Quicksort(A[s + 1..r])

Algorithm – Partition(A[l..r]) - Tracing

- 

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Partition(A[l..r]) - Tracing

- [6], 4, 3, 12, 8, 13, 7, 18.

  i i
  j i

  $3 \leq 6$ (T), $12 \leq 6$ (F)

  $12 > 6$ (T), $3 > 6$ (F)

  swap 3, 12

  [6], 4, 12, 3, 8, 13, 7, 18.

  j i

  swap 12, 3.

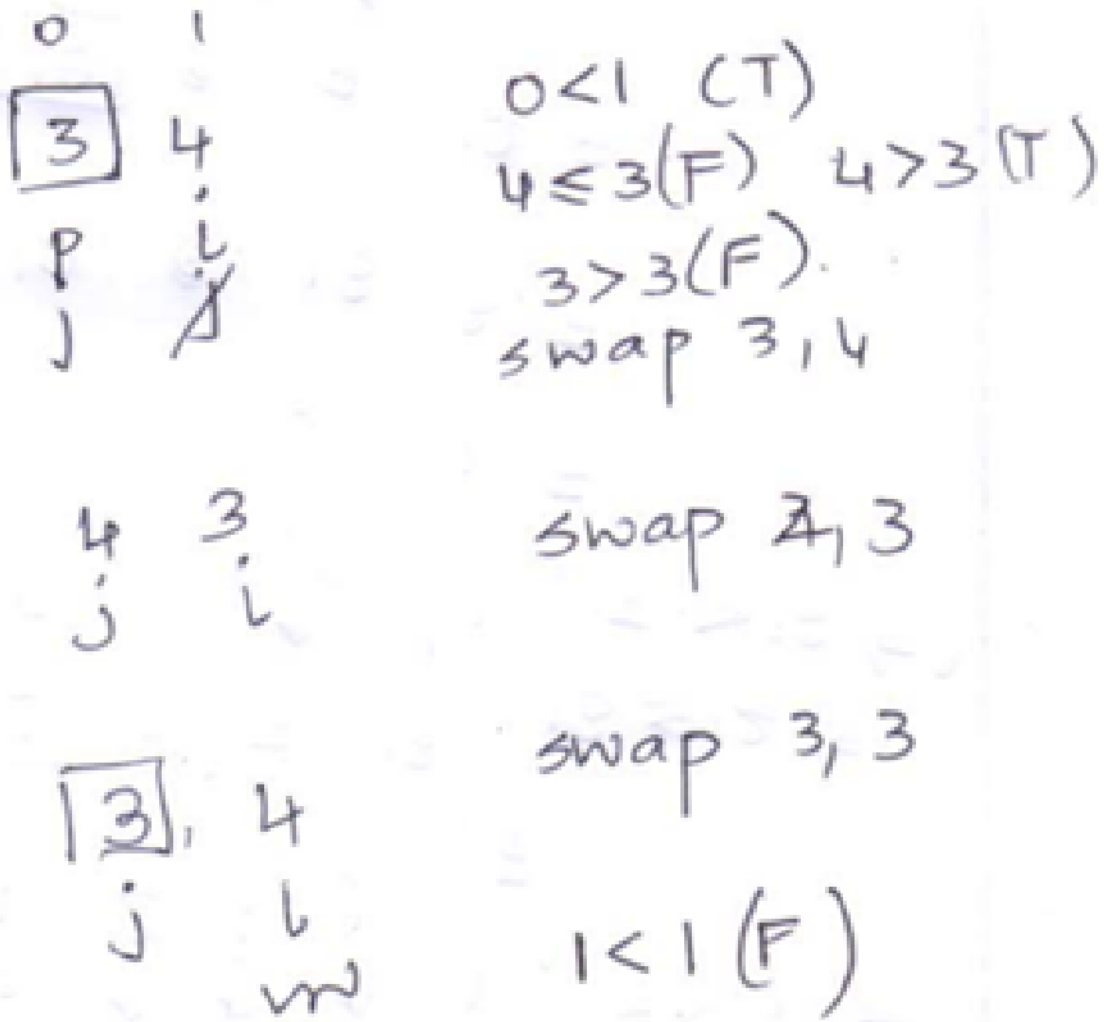
  [6], 4, 3, 12, 8, 13, 7, 18

  j i

  swap 6 and 3.

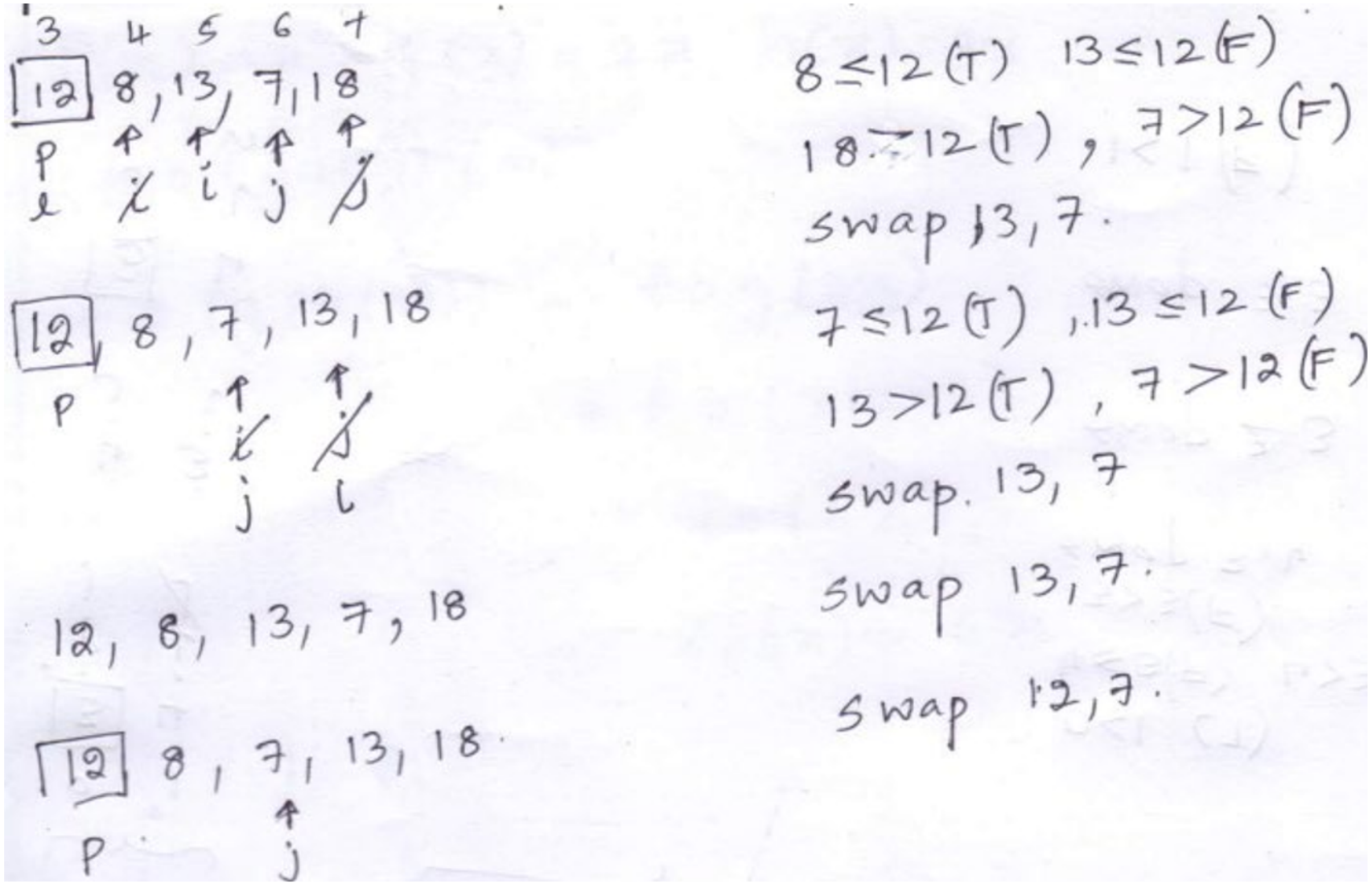
  3, 4, [6], 12, 8, 13, 7, 18

Algorithm – Partition(A[l..r]) - Tracing

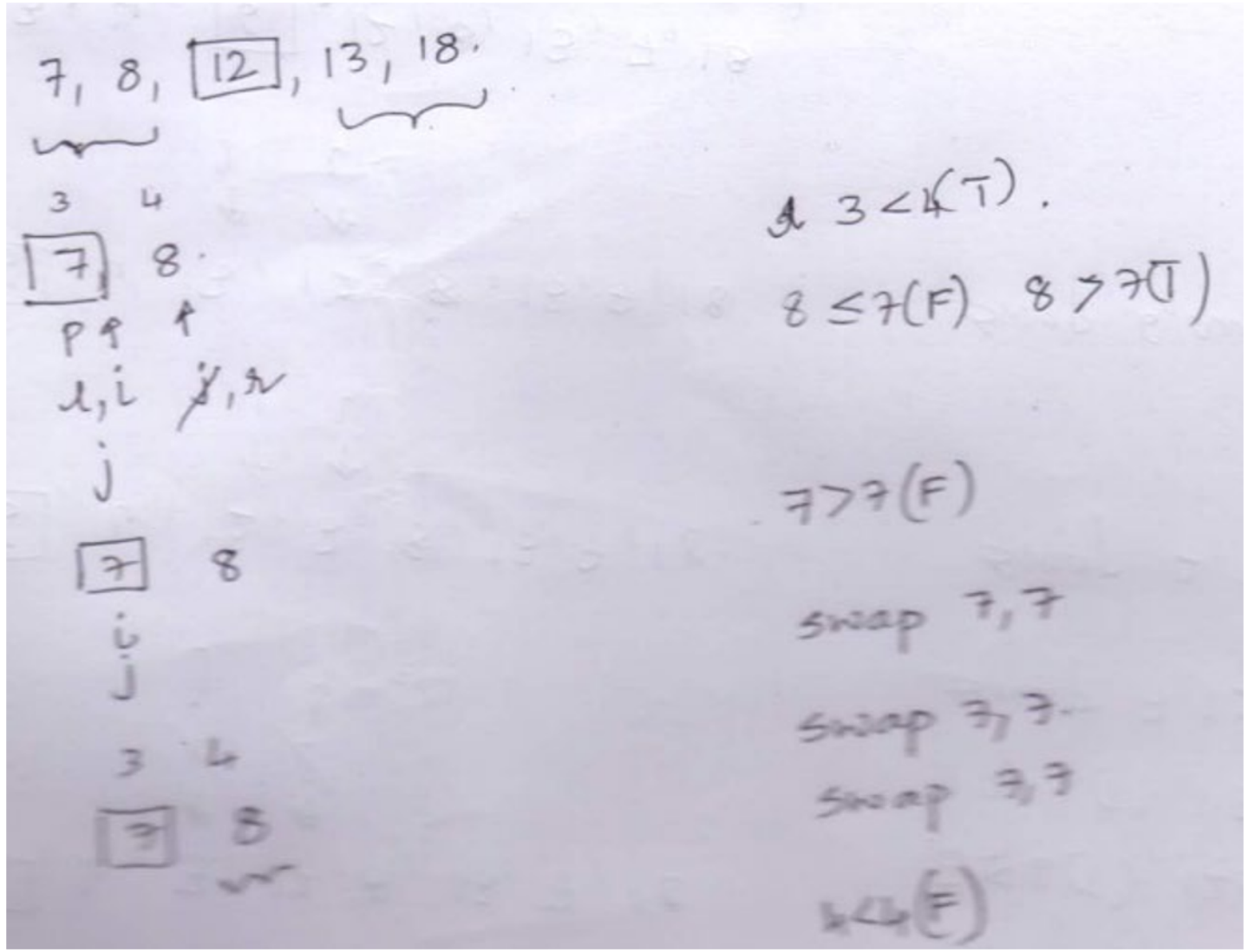
- 

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Partition(A[l..r]) - Tracing

- 

ALGORITHMS ANALYSIS AND DESIGN
Algorithm – Partition(A[l..r]) - Tracing

- 

ALGORITHMS ANALYSIS AND DESIGN

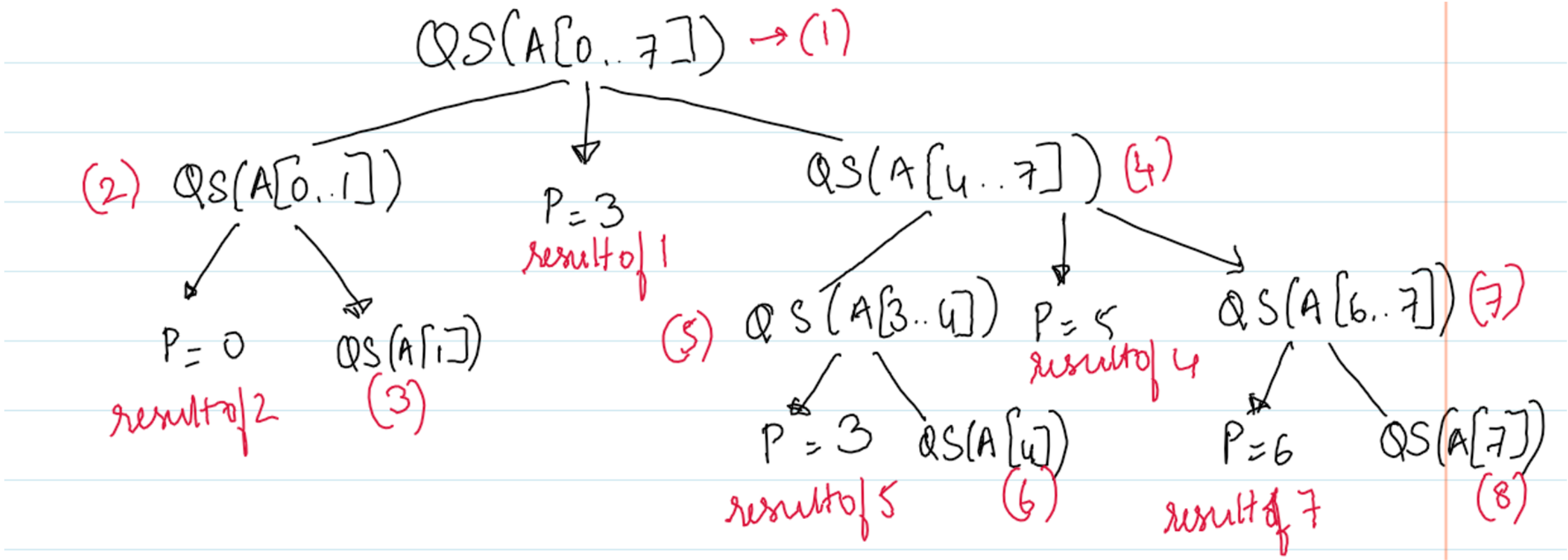
Algorithm – Partition(A[l..r]) - Tracing

- 6 7 — 6<7 (T)
  13, 18 — 18≤13(F) 18>13(T) 13>13(F)
  p, l r — Swap 13,18
  j i, ~~j~~
  18 13 — swap 13,18
  j i
  13 18 — Swap 13,13
  j i
  6 7
  13 18 — 7<7 (F)

  3, 4, 6, 7, 8, 12, 13, 18

ALGORITHMS ANALYSIS AND DESIGN

Recursive Tree



ALGORITHMS ANALYSIS AND DESIGN
Merge Sort

PES UNIVERSITY
CELEBRATING 50 YEARS

- It was created by John von Neumann in 1945.

Fig 3: https://en.wikipedia.org/wiki/John_von_Neumann

ALGORITHMS ANALYSIS AND DESIGN

Merge Sort

PES UNIVERSITY
CELEBRATING 50 YEARS

- It was created by John von Neumann in 1945.
- It sorts a given array A[0..n-1] by dividing it into two halves A[0..(n/2)-1] and A[n/2..n-1].
- Sort each of them recursively
- Merge the two smaller sorted arrays into a single sorted one.

ALGORITHMS ANALYSIS AND DESIGN
Algorithm – Mergesort(A[0..n-1])

- //Sorts array A[0..n-1] by recursive merge sort
- //Input: An array A[0..n-1] of orderable elements
- //Output: Array A[0..n-1] sorted in non-decreasing order

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Mergesort(A[0..n-1])

- if n > 1
- copy A[0..(n/2)-1] to B[0..(n/2)-1]
- copy A[(n/2)..n-1] to C[0..(n/2)-1]
- Mergesort(B[0..(n/2)-1])
- Mergesort(C[0..(n/2)-1])
- Merge (B,C,A)

ALGORITHMS ANALYSIS AND DESIGN
Algorithm – Mergesort(A[0..n-1]) - Methodology

- This can be done as follows
  - Two pointers (array indices) are initialized to point to the first elements of the arrays being merged.
  - Then the value of the index compared and smaller of them is added to new array being constructed.

ALGORITHMS ANALYSIS AND DESIGN
Algorithm – Mergesort(A[0..n-1]) - Methodology

- Now the index of the smaller element array is incremented to point to its immediate successor in the array it was copied from.
- This operation is continued until one of the two given arrays is exhausted.
- The elements of the remaining array will be copied to the end of the new array.

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Merge(B[0..P-1], C[0..Q-1],A[0..P+Q-1])

- // Merges two sorted arrays into one sorted array
- //Input: Arrays B[0..P-1] and C[0..Q-1], both sorted
- //Output: Sorted array A[0..P+Q-1] of the elements B and C

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Merge(B[0..P-1], C[0..Q-1],A[0..P+Q-1])

PES UNIVERSITY
CELEBRATING 50 YEARS

```
i←0, j ←0, k ← 0

while i<p and j < q

    if B[i] ≤ C[j]

        A[k] ← B[i], i ← i+1

    else

        A[k] ← C[j], j ← j + 1

    k ← k + 1
```

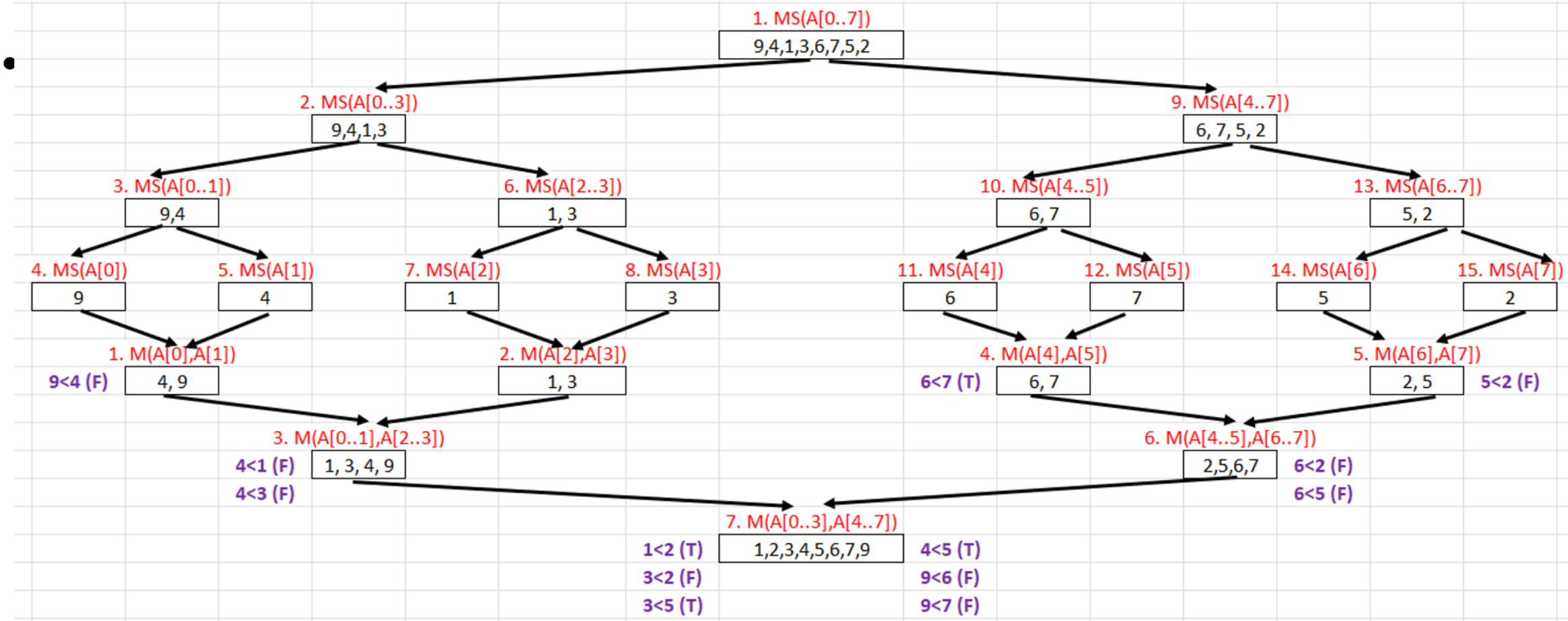
ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Merge(B[0..P-1], C[0..Q-1],A[0..P+Q-1])

```
if i = p
    copy C[j..q-1] to A[k..p+q-1]
else
    copy B[i..p-1] to A[k..p+q-1]
```

ALGORITHMS ANALYSIS AND DESIGN

Algorithm – Merge Sort - Tracing



ALGORITHMS ANALYSIS AND DESIGN

Recap

PES UNIVERSITY
CELEBRATING 50 YEARS

- Quick Sort
- Merge Sort
  - Algorithm
  - Tracing

PES UNIVERSITY

CELEBRATING 50 YEARS

THANK YOU

Lekha A
Computer Applications